Gujarati News Article Classification — Architecture & Design

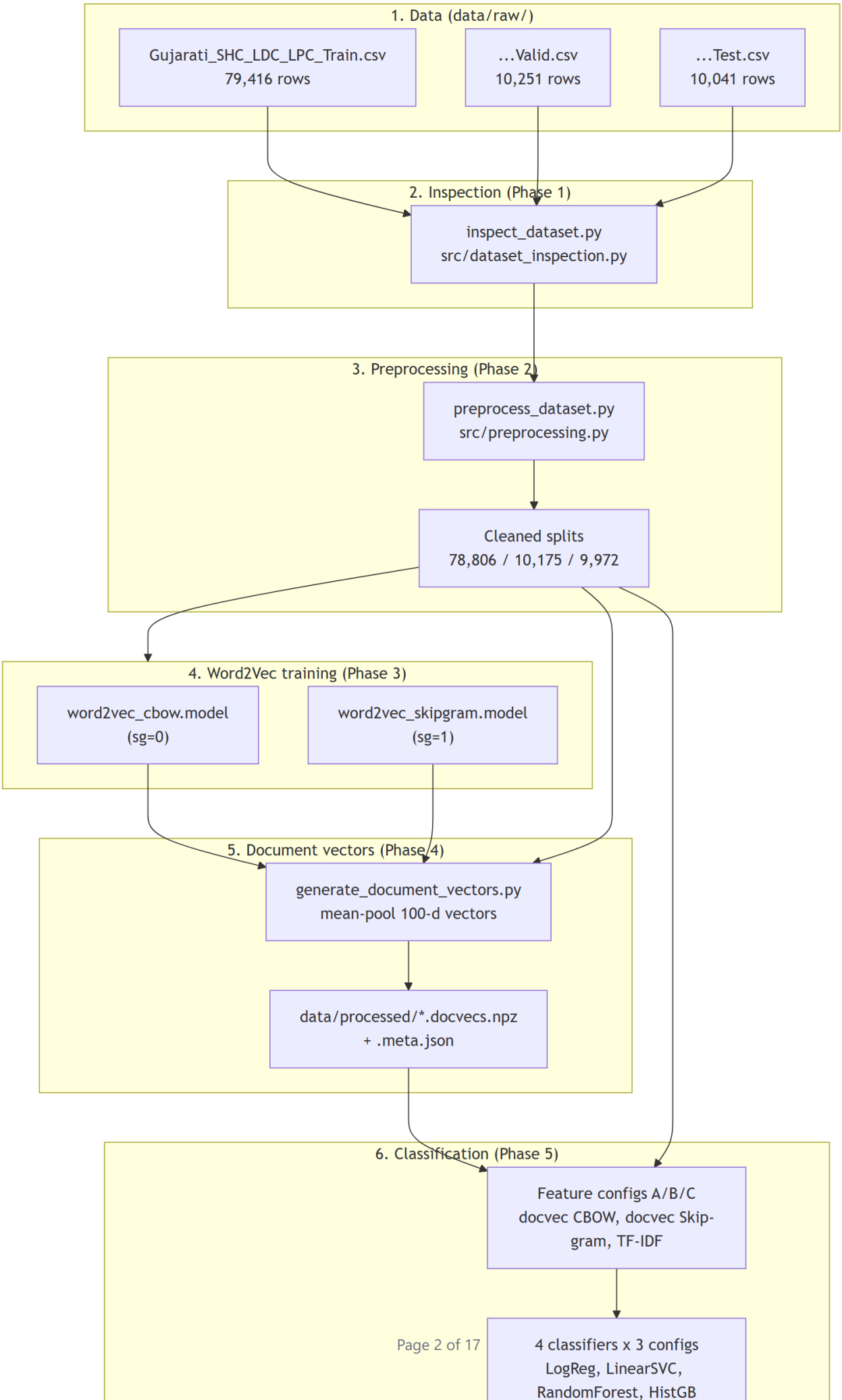
A design-rationale companion to the Project Handbook.

This document explains *how the system is built* and *why each choice was made*. It is intended for maintainers and reviewers who need to reason about the architecture, extend it, or evaluate its soundness. Every statement is grounded in the repository; anything not directly verifiable is marked *inferred* or *unverified*.

---

1. Architectural overview

The project is a **classical NLP pipeline** with a clean separation between **processes** (scripts), **library modules** (`src/`), and **presentation** (`app/`). It deliberately avoids deep learning, external services, and network calls at inference time.


1. Data (data/raw/)
Gujarati_SHC_LDC_LPC_Train.csv
79,416 rows
...Valid.csv
10,251 rows
...Test.csv
10,041 rows
2. Inspection (Phase 1)
inspect_dataset.py
src/dataset_inspection.py
3. Preprocessing (Phase 2)
preprocess_dataset.py
src/preprocessing.py
Cleaned splits
78,806 / 10,175 / 9,972
4. Word2Vec training (Phase 3)
word2vec_cbow.model
(sg=0)
word2vec_skipgram.model
(sg=1)
5. Document vectors (Phase 4)
generate_document_vectors.py
mean-pool 100-d vectors
data/processed/*.docvecs.npz
+ .meta.json
6. Classification (Phase 5)
Feature configs A/B/C
docvec CBOW, docvec Skip-gram, TF-IDF
4 classifiers x 3 configs
LogReg, LinearSVC, RandomForest, HistGB

Layering

```
+-------------------------------------------------------------+
|  Presentation        app/streamlit_app.py (view)            |
|                      app/content.py (content + pure helpers) |
+-------------------------------------------------------------+
|  Library (engine)    src/*.py                               |
|    inference facade  src/prediction.py                      |
|    modelling         src/classification.py, text_features.py |
|    understanding     src/word2vec_trainer.py, document_vectors.py
|    data handling     src/data_loader.py, preprocessing.py   |
|    reporting         src/evaluation.py, error_analysis.py,  |
|                      experiments.py, dataset_inspection.py  |
+-------------------------------------------------------------+
|  Configuration       config.py (single source of truth)     |
+-------------------------------------------------------------+
|  Processes           inspect_dataset.py ... run_model_improvement.py
+-------------------------------------------------------------+
|  Artifacts           data/, models/, reports/               |
+-------------------------------------------------------------+
```

Design principle: one responsibility per layer. The scripts orchestrate; the library computes; the GUI presents; `config.py` defines truth.

---

2. Data contracts

2.1 On-disk dataset contract

Constant	Value	Meaning
`DATA_FORMAT`	`"SHC"`	active dataset flavour
`TEXT_COLUMN`	`"text"`	the Gujarati headline column
`LABEL_COLUMN`	`"labels"`	the category column
`TRAIN_ENCODING`	`"utf-8-sig"`	BOM-tolerant UTF-8

The loader (`src/data_loader.py`) produces a `DatasetSplits` object (train / valid / test) and runs **structural validation** (`validate_structure`, `validate_all`) so that a malformed file fails fast with a clear `DataLoaderError`.

> **Note:** the CSV filenames encode `SHC_LDC_LPC`, but the repository's actual CSVs contain only the `text` and `labels` columns — i.e. the **SHC** shape. Alternate formats are recognised by the loader but not present. See the audits in `PROJECT_AUDIT.md`.

2.2 Feature artifact contract

Each document-vector file is a NumPy `.npz` with three arrays plus a JSON sidecar:

Key	Shape / type	Purpose
`X`	`(rows, 100)` float32	the feature matrix
`labels`	`(rows,)`	category per row
`row_ids`	`(rows,)` int	positional identity for alignment

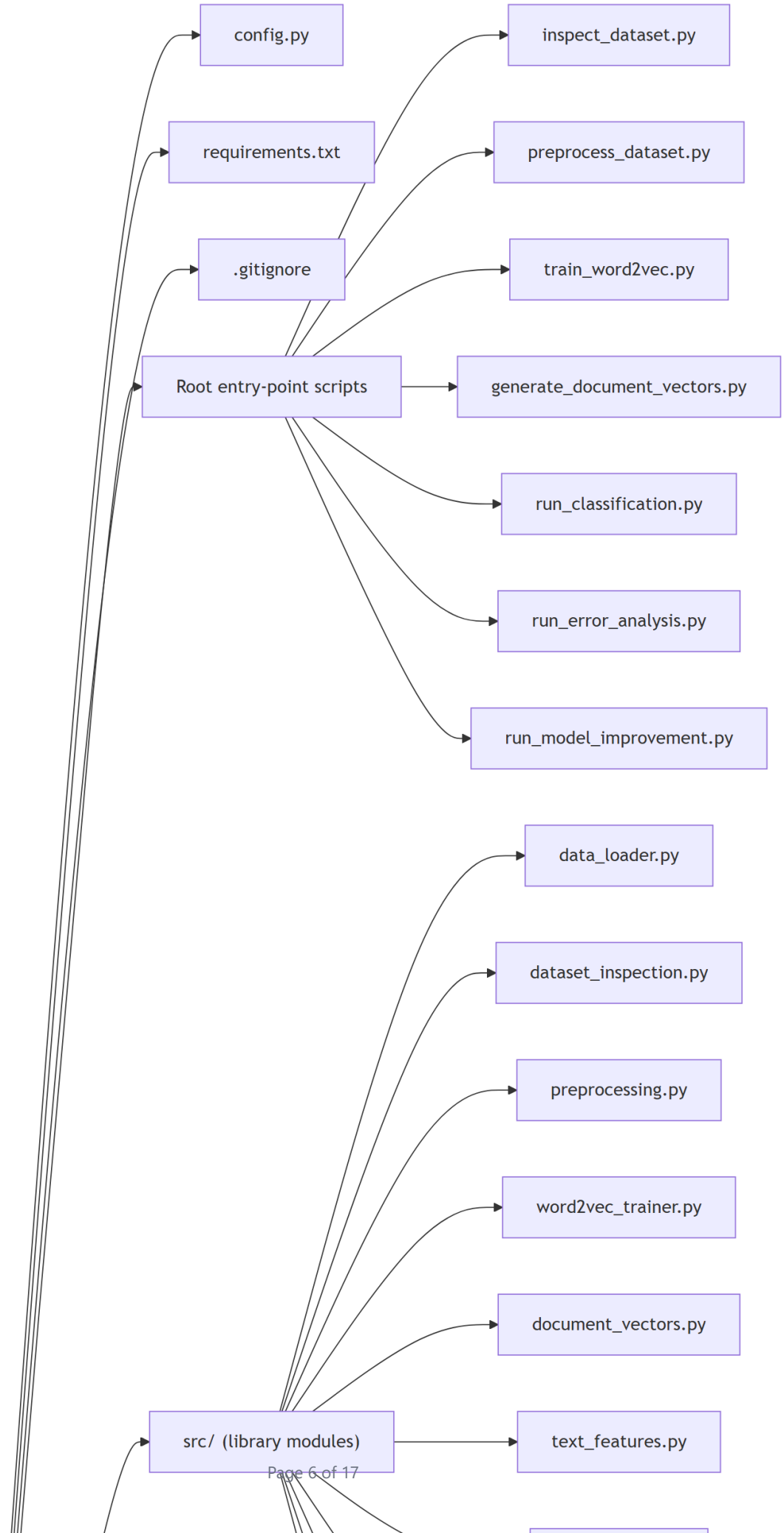
The metadata (`<name>.<model>.docvecs.meta.json`) records model name, `sg`, dimension, preprocessing method, source split/file, row count, fallback count and fraction, empty-token count, OOV/in-vocab token totals, mean in-vocab tokens, vocab size, and the absolute model path.

2.3 Classifier artifact contract

`models/classifiers/selected_model.joblib` is a dictionary bundle produced by `save_classifier_artifact(...)` and consumed by `load_classifier_artifact(...)`. It contains the fitted estimator **plus** the metadata needed to rebuild features at inference: feature type, class names, and (for TF-IDF models) the fitted vectorizer and SVD transformer. `src/prediction.NewsClassifier` is the only supported reader.

---

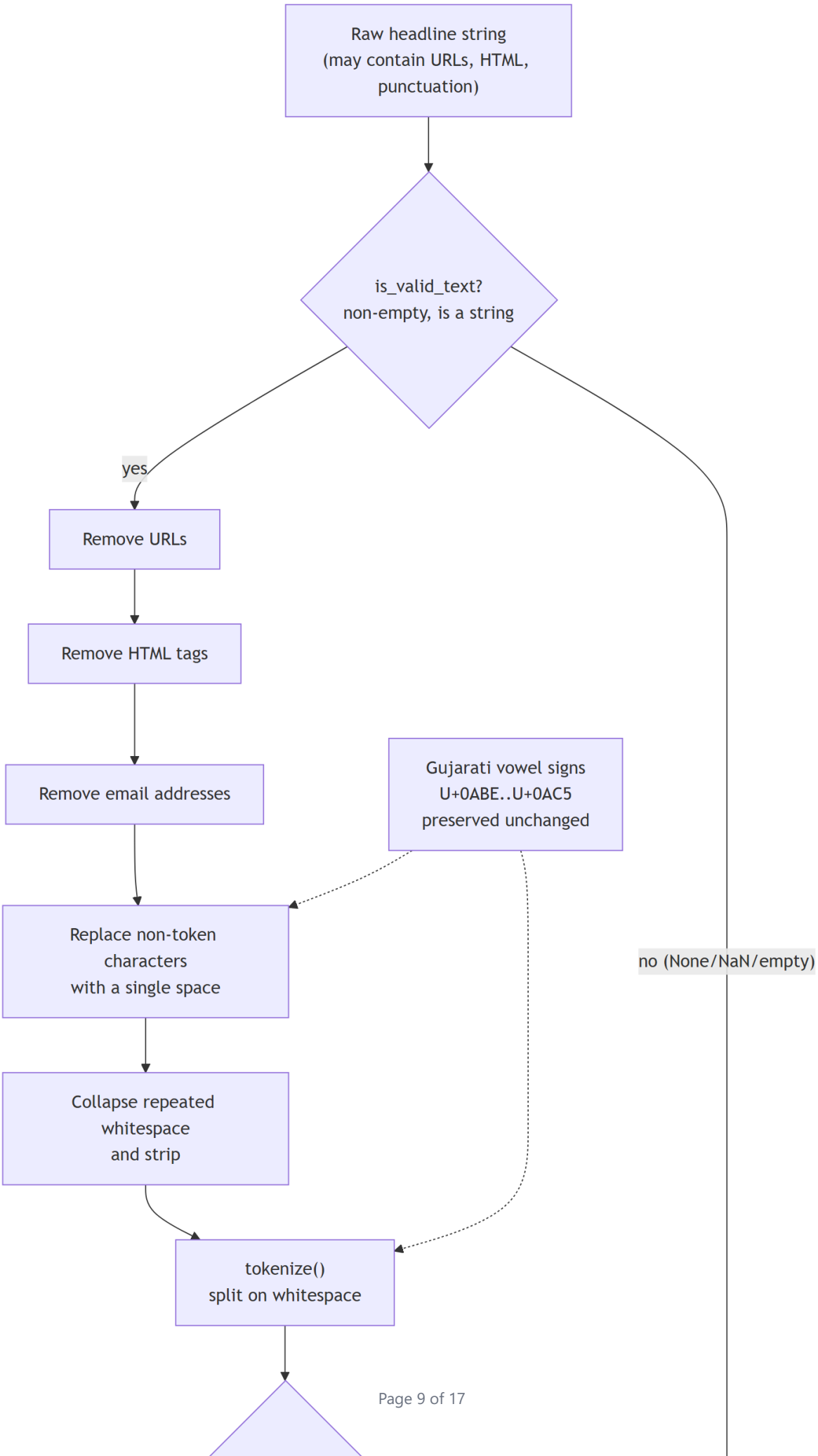
3. Component responsibilities


config.py
requirements.txt
.gitignore
Root entry-point scripts
inspect_dataset.py
preprocess_dataset.py
train_word2vec.py
generate_document_vectors.py
run_classification.py
run_error_analysis.py
run_model_improvement.py
data_loader.py
dataset_inspection.py
preprocessing.py
word2vec_trainer.py
document_vectors.py
src/ (library modules)
text_features.py

Component	Inputs	Outputs	Key design choice
`config.py`	—	constants	one source of paths/constants
`data_loader.py`	raw CSVs	`DatasetSplits`	fail-fast structural validation
`dataset_inspection.py`	`DatasetSplits`	inspection report	read-only reconnaissance
`preprocessing.py`	text	cleaned text + tokens	conservative, Gujarati-safe, deterministic
`word2vec_trainer.py`	cleaned train tokens	two `.model` files, report	labels never used (unsupervised)
`document_vectors.py`	tokens + model	`.npz` + `.meta.json`, report	OOV-skipping mean pooling
`text_features.py`	docvec `.npz`, cleaned text	feature bundles	TF-IDF fitted on train only
`evaluation.py`	predictions	metrics/tables/plots	macro-F1 as headline metric
`classification.py`	feature bundles	experiments, selected model	grid + select on validation
`prediction.py`	headline string	label + scores	single inference facade
`error_analysis.py`	selected model + validation	error reports	validation-only analysis
`experiments.py`	feature bundles	Phase 7 experiments + decision	pre-registered replacement threshold
`app/streamlit_app.py`	user input	rendered UI	view only, delegates inference
`app/content.py`	—	content + helpers	importable without Streamlit

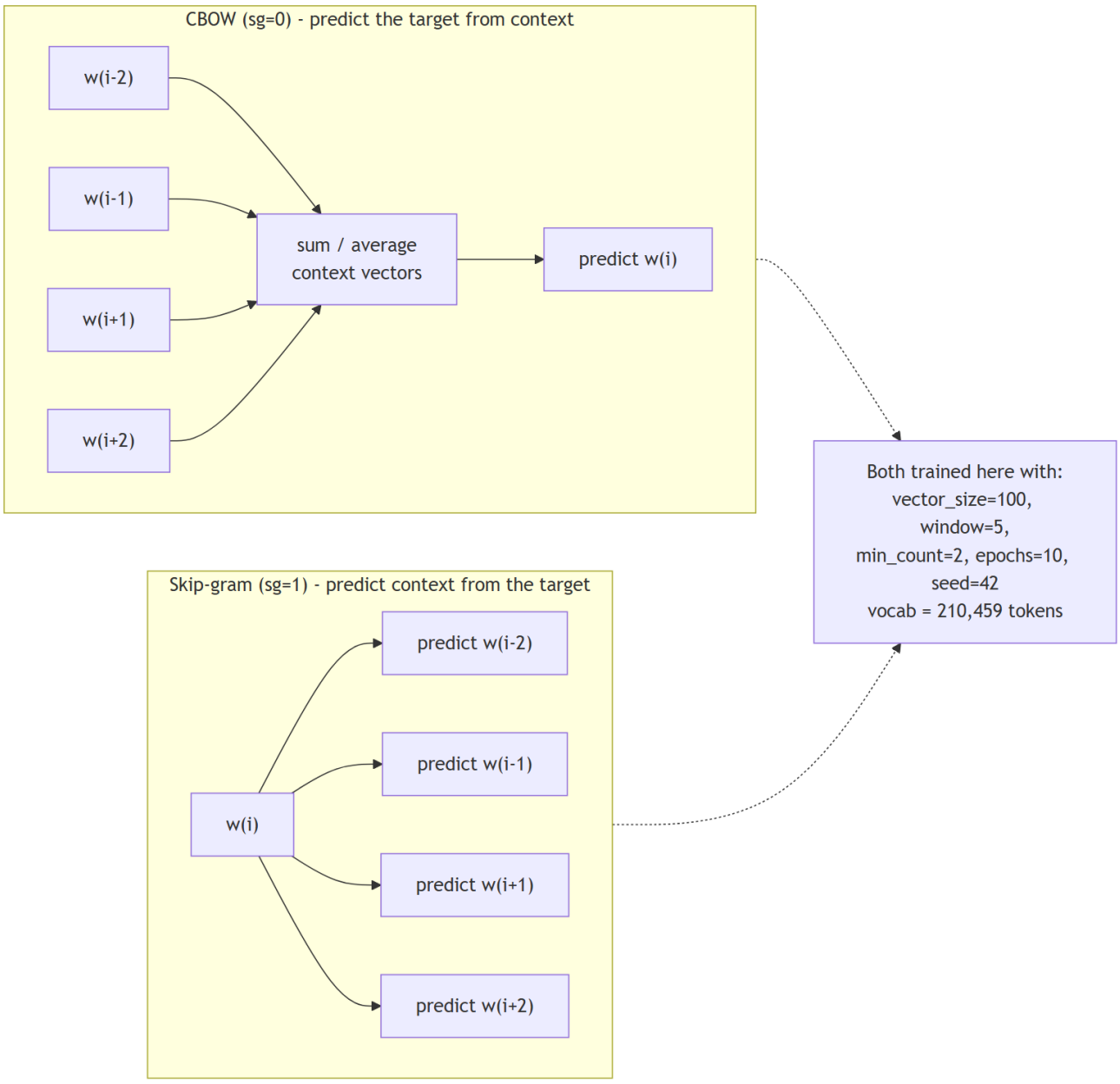
4. The pipeline, stage by stage

4.1 Preprocessing (Phase 2)


Raw headline string
(may contain URLs, HTML,
punctuation)
is_valid_text?
non-empty, is a string
yes
no (None/NaN/empty)
Remove URLs
Remove HTML tags
Remove email addresses
Gujarati vowel signs
U+0ABE..U+0AC5
preserved unchanged
Replace non-token
characters
with a single space
Collapse repeated
whitespace
and strip
tokenize()
split on whitespace

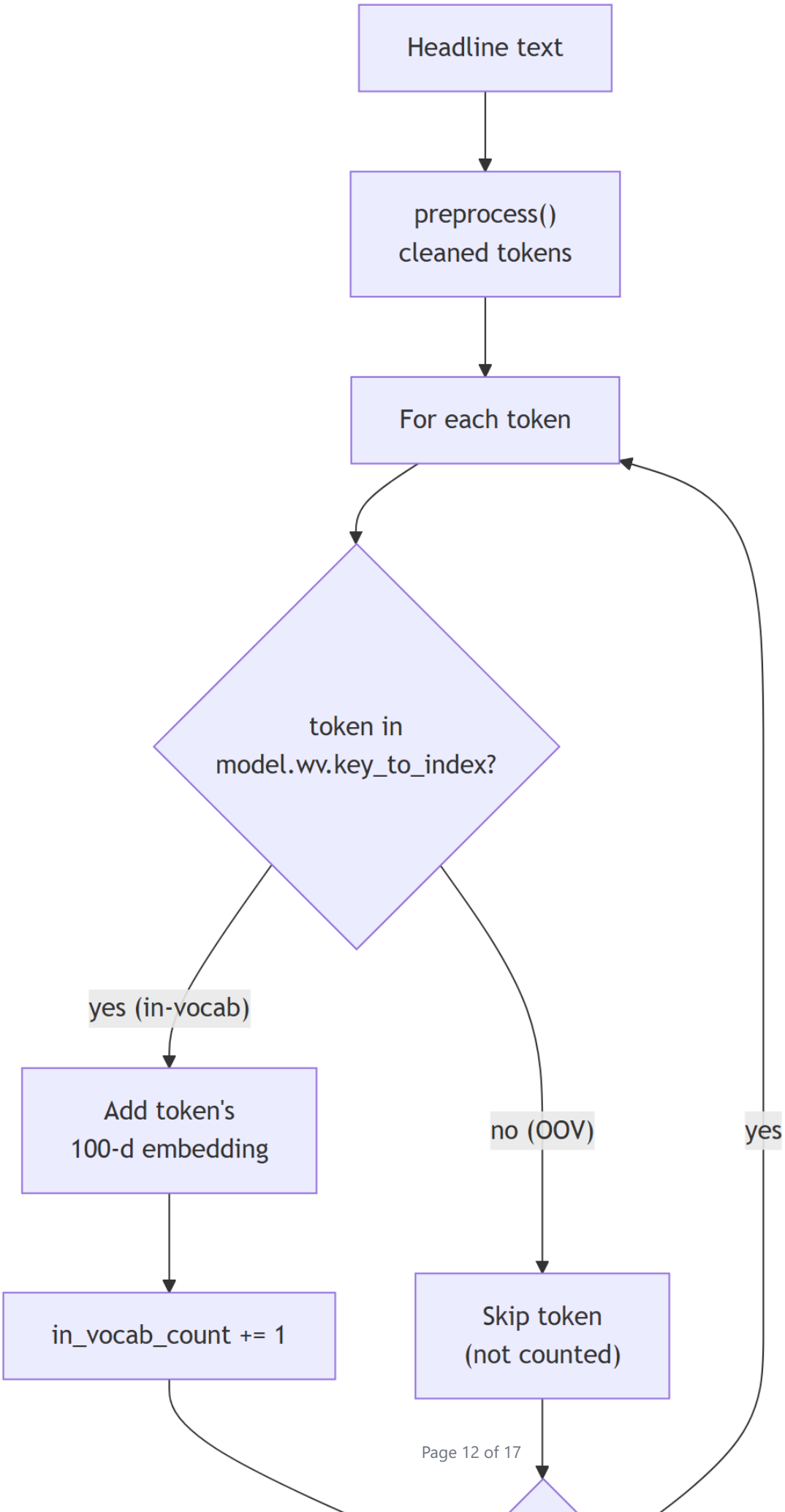
Order matters: URL → HTML → email removal happen **before** punctuation-to-space, so removed markup cannot leak characters into tokens. Punctuation is replaced with a **space, not deleted**, preventing accidental word merges. The function is pure and idempotent so it behaves identically at training and inference time — the single most important consistency guarantee in the system.

4.2 Embeddings (Phase 3)



Two unsupervised models are trained on the train split only, with identical hyper-parameters except `sg`. No labels are used. Gensim is run in its deterministic (`fast version: 0`) path with `seed=42`.

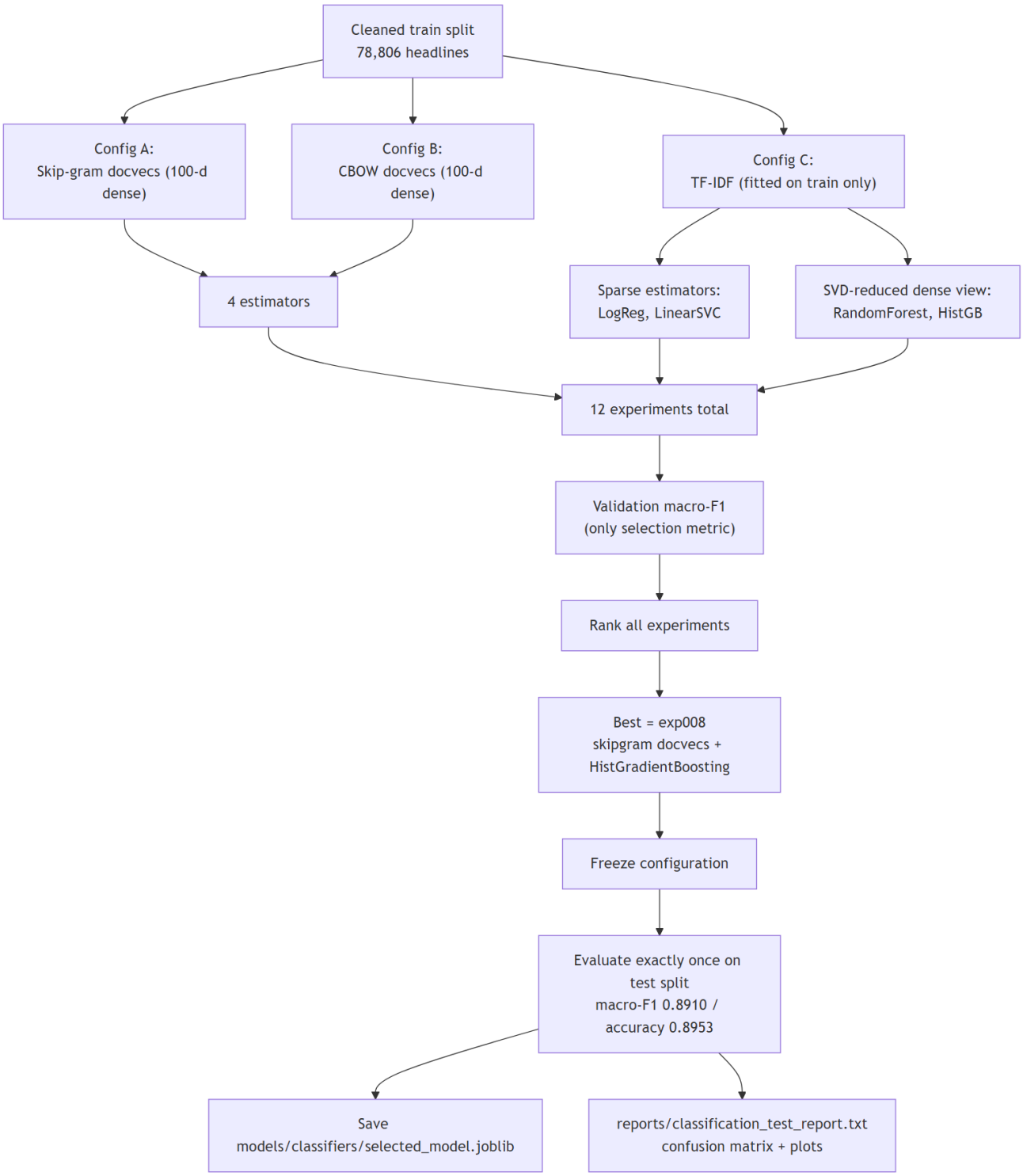
4.3 Document vectors (Phase 4)


Headline text
preprocess()
cleaned tokens
For each token
token in
model.wv.key_to_index?
yes (in-vocab)
Add token's
100-d embedding
in_vocab_count += 1
no (OOV)
Skip token
(not counted)
yes

Two rules define correctness:

1. **OOV tokens are skipped**, so the average divides by the in-vocab count, not the raw token count.
2. **All-OOV headlines** produce a zero vector and are counted as *fallbacks* rather than producing `NaN` or crashing.

4.4 Classification (Phase 5)

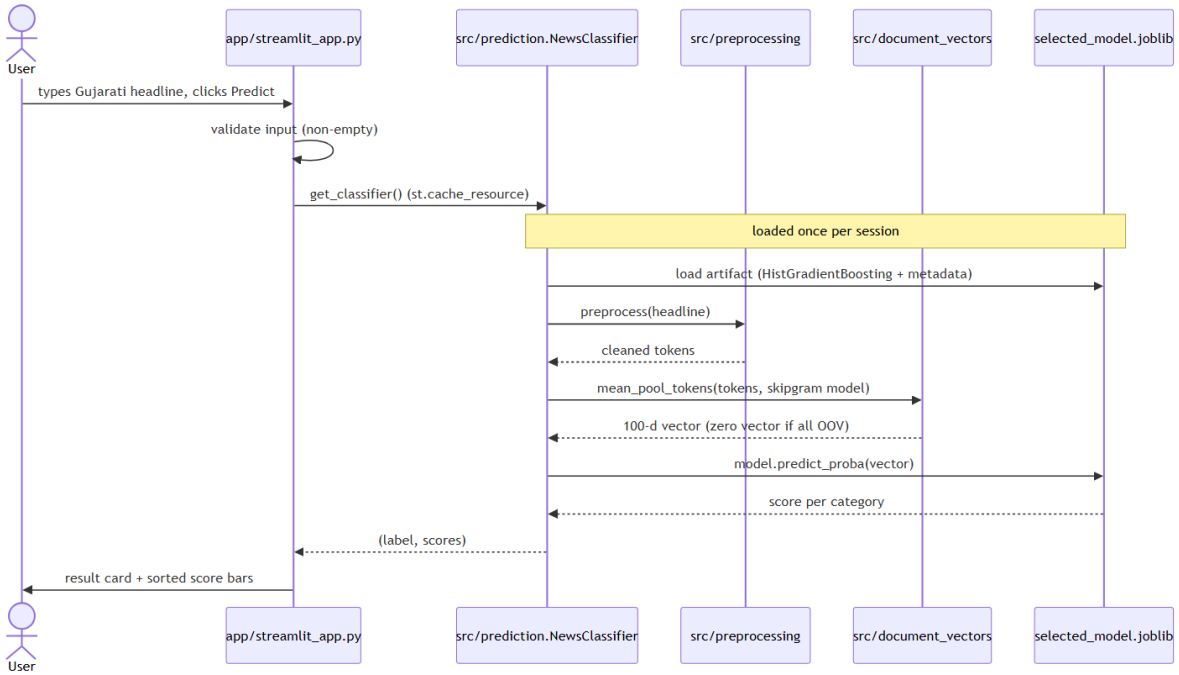


A 3×4 grid (three feature configs × four estimators) is evaluated on validation, ranked by macro-F1, and the single best configuration is frozen and evaluated once on test. Trees use an SVD-reduced dense view of TF-IDF; linear models use the raw sparse matrix.

4.5 Error analysis & improvement (Phase 7)

Baseline reproduction → error characterisation → cause investigation → probability quality → 15 improvement experiments → **retain/replace** decision against a pre-registered `REPLACEMENT_THRESHOLD = 0.005` macro-F1 margin. Outcome: **RETAIN**.

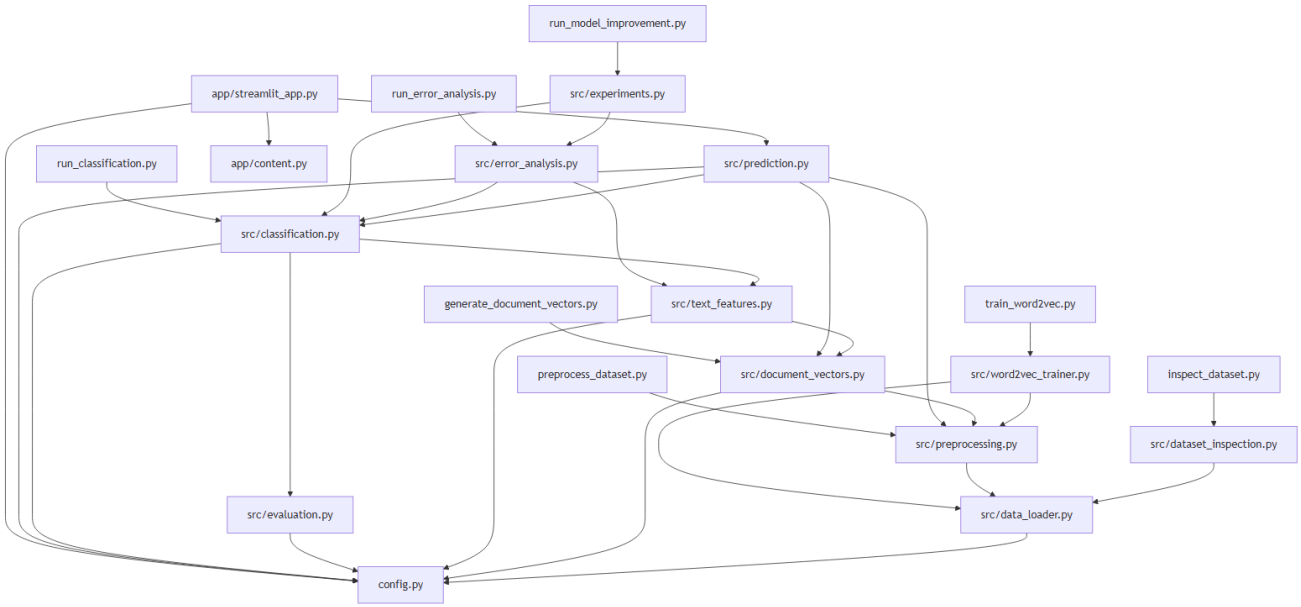
4.6 Inference (Phase 6 GUI)



The GUI loads the frozen artifact once per session (`st.cache_resource`), preprocesses the headline, mean-pools it with the skip-gram model, and calls `predict_proba`. Scores are labelled honestly as *probability estimates*.

---

5. Module dependency structure



`config.py` is the universal root. `data_loader.py` and `preprocessing.py` sit near the bottom. `prediction.py` is the top of the library and the only module the GUI imports for inference. The root scripts are thin adapters over library modules.

> **Dependency note:** the graph above reflects module imports observed in the source. `src/preprocessing.py` depends on `src/data_loader.py` for its end-to-end pipeline entry point even though the pure cleaning functions have no such dependency.

---

6. Cross-cutting concerns

6.1 Data-leakage prevention (the dominant design constraint)

Stage	Leakage guard
Word2Vec	trained on train split only; labels unused
TF-IDF	vectorizer fitted on train text only; valid/test transformed only
Model selection	ranked on validation only
Test evaluation	performed exactly once, after freezing
Phase 7	validation only; test never inspected

A dedicated test asserts that a token appearing only in validation is absent from the fitted TF-IDF vocabulary.

6.2 Determinism & reproducibility

- `RANDOM_SEED = 42` everywhere the estimator supports it.
- Gensim deterministic path (`fast version: 0`).
- Pure, idempotent preprocessing.
- Baseline reproduction matched to `0.00e+00` .
- *Caveat:* cross-machine, cross-thread bit-exactness is likely but not guaranteed (Word2Vec uses `workers=8`).

6.3 Error handling

- `data_loader` raises `DataLoaderError` on structural problems.
- `prediction` raises a clear `ValueError` on empty/whitespace input and a friendly error if the artifact is missing.
- The GUI converts missing-artifact conditions into actionable messages rather than stack traces.

6.4 Resource envelope

- Word2Vec models: ≈9.37 MB each.
- Some classifier artifacts are very large (up to ~822 MB for the TF-IDF + RandomForest experiment), because the fitted vectorizer/SVD can be embedded in the joblib bundle.
- Cleaned-text CSV dumps total ≈652 MB and are **orphaned** (not read downstream).

---

7. Extension points

To change…	Do this	Guard
Add a category	update `CATEGORY_LABELS` and retrain	metrics assume the frozen label set
Add a classifier	extend the experiment grid in `src/classification.py`	re-select on validation
Add a feature set	implement a bundle builder in `src/text_features.py`	fit on train only
Try sub-word units	replace tokenization in `src/preprocessing.py`	keep train/inference identical
Improve calibration	use the existing `calibrated_variant` in `src/experiments.py`	justify vs. added complexity
Deploy as a service	wrap `src/prediction.NewsClassifier`	ship the `.npy` sidecars

---

8. What the architecture optimises for

1. **Correctness of evaluation** — leakage discipline above all else.
2. **Explainability** — every stage is a transparent, classical technique.
3. **Reproducibility** — fixed seeds, deterministic paths, read-only discipline.
4. **Low operational cost** — no GPUs, no network, small core models.
5. **Honesty** — unverified claims are marked; scores are described accurately.

End of architecture document.